

Section 13.1 — DOM Selection & Manipulation

JavaScript দিয়ে browser-এর HTML document-এর element খুঁজে বের করা, তাদের content পরিবর্তন করা, attribute পরিবর্তন করা, class যোগ/বাদ দেওয়া এবং নতুন element তৈরি করার জন্য **DOM (Document Object Model)** ব্যবহার করা হয়।

এই document-এ আমরা শিখব:

1. DOM কী
2. DOM Tree
3. `document`
4. `querySelector()`
5. `querySelectorAll()`
6. `NodeList`
7. `getElementById()`
8. `getElementsByClassName()`
9. `HTMLCollection`
10. `NodeList` vs `HTMLCollection`
11. Live vs Static Collection
12. `innerHTML`
13. `textContent`
14. `innerText`
15. `innerHTML` vs `textContent` vs `innerText`
16. `setAttribute()`
17. `getAttribute()`
18. `removeAttribute()`
19. `hasAttribute()`
20. `classList`
21. `classList.add()`
22. `classList.remove()`
23. `classList.toggle()`
24. `classList.contains()`
25. `dataset`
26. Basic DOM traversal
27. `parentElement`
28. `children`
29. `firstElementChild`
30. `nextElementSibling`
31. `createElement()`
32. `append()`
33. `prepend()`
34. `remove()`
35. Practical DOM manipulation
36. Common mistakes
37. Best practices
38. Practice problems

1. DOM কী?

DOM = Document Object Model

Browser যখন কোনো HTML page load করে, তখন HTML document-কে একটি **object-based tree structure** হিসেবে তৈরি করে।

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello</h1>
    <p>Welcome</p>
  </body>
</html>
```

Browser এটাকে roughly এমন একটা tree হিসেবে দেখে:

```
Document
├── html
│   └── body
│       ├── h1
│       │   └── "Hello"
│       └── p
│           └── "Welcome"
```

JavaScript এই DOM tree-এর elementগুলোকে access এবং manipulate করতে পারে।

2. DOM কেন দরকার?

ধরো HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript দিয়ে আমরা করতে পারি:

```
const title = document.querySelector("#title");  
  
title.textContent = "Hello Shariar";
```

Browser-এ:

Hello

এর বদলে:

Hello Shariar

দেখাবে।

অর্থাৎ:

```
HTML  
↓  
DOM  
↓  
JavaScript  
↓  
DOM পরিবর্তন  
↓  
Browser UI পরিবর্তন
```

3. document কী?

Browser-এর current HTML document-কে represent করে `document` object।

```
console.log(document);
```

`document` থেকে আমরা HTML-এর বিভিন্ন element খুঁজে বের করতে পারি।

Example:

```
document.querySelector("h1");
```

এখানে:

```
document
  ↓
querySelector()
  ↓
HTML element
```

4. querySelector()

`querySelector()` একটি CSS selector ব্যবহার করে প্রথম **matching element** return করে।

HTML:

```
<h1 id="title">Hello</h1>
<p>Paragraph 1</p>
<p>Paragraph 2</p>
```

JavaScript:

```
const title = document.querySelector("#title");

console.log(title);
```

4.1 Tag selector

```
const heading = document.querySelector("h1");
```

এটি প্রথম `<h1>` return করে।

যদি থাকে:

```
<h1>One</h1>
<h1>Two</h1>
```

তাহলে:

```
document.querySelector("h1");
```

শুধু প্রথম `<h1>` পাবে।

5. Class selector

HTML:

```
<div class="card">Card 1</div>  
<div class="card">Card 2</div>
```

JavaScript:

```
const card = document.querySelector(".card");
```

শুধু প্রথম `.card` element পাওয়া যাবে।

6. ID selector

HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript:

```
const title = document.querySelector("#title");
```

7. Attribute selector

CSS selector ব্যবহার করা যায় বলে অনেক powerful selection করা সম্ভব।

```
<input type="text">  
<input type="email">
```

JavaScript:

```
const emailInput = document.querySelector('input[type="email"]');
```

8. Nested selector

HTML:

```
<div class="container">  
  <p>Hello</p>  
</div>
```

JavaScript:

```
const paragraph = document.querySelector(".container p");
```

এখানে `.container` এর ভিতরের `<p>` select করা হচ্ছে।

9. `querySelector()` না পেলে কী **return** করে?

যদি matching element না পাওয়া যায়:

```
const element = document.querySelector(".does-not-exist");  
  
console.log(element);
```

Output:

```
null
```

তাই পরে সরাসরি:

```
element.textContent = "Hello";
```

করলে **error** হতে পারে।

Safe approach:

```
if (element) {  
  element.textContent = "Hello";  
}
```

10. `querySelectorAll()`

`querySelector()` যেখানে প্রথম **element** return করে, `querySelectorAll()` সেখানে সব **matching elements** return করে।

HTML:

```
<p class="text">One</p>  
<p class="text">Two</p>  
<p class="text">Three</p>
```

JavaScript:

```
const paragraphs = document.querySelectorAll(".text");  
  
console.log(paragraphs);
```

এটি একটি **NodeList** return করবে।

11. **NodeList**

`querySelectorAll()` সাধারণত একটি **NodeList** return করে।

```
const items = document.querySelectorAll(".item");
```

NodeList-এর মধ্যে multiple DOM nodes থাকতে পারে।

NodeList loop করা

```
const items = document.querySelectorAll(".item");

items.forEach((item) => {
  console.log(item.textContent);
});
```

12. `querySelector()` vs `querySelectorAll()`

Method	Result
<code>querySelector()</code>	প্রথম matching element
<code>querySelectorAll()</code>	সব matching elements
Not found	<code>null</code>
<code>querySelectorAll()</code> no match	empty NodeList

Example:

```
const first = document.querySelector(".item");

const all = document.querySelectorAll(".item");
```

Mental model:

```
querySelector
  ↓
first match

querySelectorAll
  ↓
all matches
```

13. Nested `querySelector()`

একটি element-এর ভিতরেও `querySelector()` ব্যবহার করা যায়।


```
<div class="card">
  <h2>Product</h2>
  <p>Price: $100</p>
</div>
```

```
const card = document.querySelector(".card");

const title = card.querySelector("h2");

console.log(title.textContent);
```

Output:

Product

এটি বড় DOM-এর মধ্যে নির্দিষ্ট অংশ নিয়ে কাজ করার জন্য useful।

14. getElementById()

ID দিয়ে element select করার পুরনো এবং straightforward method:

```
<h1 id="title">Hello</h1>
```

```
const title = document.getElementById("title");
```

দিতে হবে না

এখানে:

```
document.getElementById("title");
```

সঠিক।

এটি ভুল:

```
document.getElementById("#title");
```

কারণ `getElementById()` সরাসরি ID নেয়।

15. `querySelector("#title")` vs `getElementById("title")`

দুটোই ID select করতে পারে:

```
document.querySelector("#title");
```

```
document.getElementById("title");
```

Difference:

```
querySelector()  
→ CSS selector  
  
getElementById()  
→ directly ID
```

`querySelector()` বেশি flexible:

```
document.querySelector(".card");  
document.querySelector("button");  
document.querySelector("#title");  
document.querySelector("input[type='email']");
```

16. `getElementsByName()`

Class name দিয়ে multiple elements select করতে:

```
<p class="text">One</p>  
<p class="text">Two</p>
```

```
const elements = document.getElementsByClassName("text");
```

এটি একটি **HTMLCollection** return করে।

17. HTMLCollection

`getElementsByClassName()` সাধারণত HTMLCollection return করে।

```
const cards = document.getElementsByClassName("card");
```

এখানে:

```
HTMLCollection
├── card
├── card
└── card
```

18. NodeList vs HTMLCollection

দুটো দেখতে অনেকটা similar হলেও technically আলাদা।

Feature	NodeList	HTMLCollection
<code>querySelectorAll()</code>	✓	✗
<code>getElementsByClassName()</code>	✗	✓
<code>forEach()</code>	সাধারণত ✓	সব ক্ষেত্রে নয়
Live হতে পারে?	সাধারণত static	সাধারণত live
শুধু Element?	সবসময় নয়	হ্যাঁ

সবচেয়ে গুরুত্বপূর্ণ:

```
querySelectorAll()
```

→ static NodeList

```
getElementsByClassName()
```

→ live HTMLCollection

19. Live Collection vs Static Collection

এটি DOM-এর একটি important concept।

ধরো:

```
<div class="item">One</div>  
<div class="item">Two</div>
```

```
const items = document.getElementsByClassName("item");  
  
console.log(items.length);
```

Output:

2

এখন নতুন element add করলাম:

```
const newItem = document.createElement("div");  
  
newItem.classList.add("item");  
  
document.body.append(newItem);
```

তারপর:

```
console.log(items.length);
```

Output হবে:

3

কারণ **HTMLCollection** **live**।

অর্থাৎ DOM পরিবর্তন হলে **collection**-ও **update** হয়।

Static NodeList

```
const items = document.querySelectorAll(".item");
```

এটি সাধারণত একটি **snapshot**-এর মতো কাজ করে।

পরে DOM পরিবর্তন করলেও আগের **NodeList** নিজে থেকে **update** হবে না।

Mental model:

```
HTMLCollection  
→ live view  
  
querySelectorAll()  
→ static snapshot
```

20. innerHTML

Element-এর ভিতরের **HTML content** read বা write করতে **innerHTML** ব্যবহার করা যায়।

HTML:

```
<div id="box"></div>
```

JavaScript:

```
const box = document.querySelector("#box");  
  
box.innerHTML = "<h2>Hello</h2>";
```

Browser-এ **<h2>** element তৈরি হয়ে যাবে।

21. `innerHTML` দিয়ে **HTML** তৈরি

```
box.innerHTML = `  
  <h2>Product</h2>  
  <p>Price: $100</p>  
`;  
;
```

এখানে HTML string parse হয়ে DOM element তৈরি করবে।

22. `innerHTML` দিয়ে **content read**

```
<div id="box">  
  <h2>Hello</h2>  
</div>
```

```
const box = document.querySelector("#box");  
  
console.log(box.innerHTML);
```

Output roughly:

```
<h2>Hello</h2>
```

23. `textContent`

`textContent` element-এর text content read/write করার জন্য ব্যবহার করা হয়।

```
<div id="box">  
  <h2>Hello</h2>  
</div>
```

```
const box = document.querySelector("#box");  
  
console.log(box.textContent);
```

Output:

Hello

24. `textContent` দিয়ে `text` set

```
box.textContent = "Hello Shariar";
```

এটি plain text হিসেবে content set করবে।

যদি লিখি:

```
box.textContent = "<h2>Hello</h2>";
```

Browser `<h2>` render করবে না।

বরং text হিসেবে দেখাবে:

```
<h2>Hello</h2>
```

25. `innerHTML` vs `textContent`

```
element.innerHTML = "<strong>Hello</strong>";
```

এখানে HTML parse হবে।

কিন্তু:

```
element.textContent = "<strong>Hello</strong>";
```

এখানে plain text থাকবে।

Property	HTML parse করে?	Text
<code>innerHTML</code>	✓	HTML + text

Property	HTML parse করে?	Text
textContent	✗	সব text
innerText	✗	visible text

26. innerText

innerText user-এর কাছে visually rendered text-এর সাথে বেশি সম্পর্কিত।

Example:

```
<div>
  Hello
  <span style="display: none;">Hidden</span>
</div>
```

```
const div = document.querySelector("div");
console.log(div.innerText);
```

সাধারণত hidden content innerText -এ ধরা পড়বে না।

কিন্তু:

```
console.log(div.textContent);
```

hidden text-ও include করতে পারে।

27. innerText vs textContent

textContent

DOM-এর text content নিয়ে কাজ করে।

innerText

Rendered/visible text-এর সাথে বেশি সম্পর্কিত।

Practical rule:

Plain text read/write
→ `textContent`

HTML inject করতে
→ `innerHTML`

Rendered text দরকার হলে
→ `innerText`

28. `innerHTML` এবং Security

এখানে একটি খুব important security issue আছে:

XSS = Cross-Site Scripting

যদি untrusted user input সরাসরি `innerHTML` -এ ঢোকানো হয়, security problem হতে পারে।

Example:

```
const userInput = getUserInput();  
  
element.innerHTML = userInput;
```

User যদি malicious HTML/script-like content দেয়, সেটি dangerous হতে পারে।

তাই user-provided plain text হলে:

```
element.textContent = userInput;
```

ব্যবহার করা safer।

29. `setAttribute()`

HTML element-এর attribute set করতে:

```
<a id="link">Google</a>
```

```
const link = document.querySelector("#link");  
  
link.setAttribute("href", "https://example.com");
```

Common attributes

```
element.setAttribute("id", "title");  
  
element.setAttribute("title", "Hello");  
  
element.setAttribute("disabled", "");  
  
element.setAttribute("data-id", "123");
```

30. `getAttribute()`

Attribute-এর value read করতে:

```
const value = element.getAttribute("title");  
  
console.log(value);
```

Example:

```
<button id="btn" title="Click me">  
  Click  
</button>
```

```
const button = document.querySelector("#btn");  
  
console.log(button.getAttribute("title"));
```

Output:

Click me

31. removeAttribute()

Attribute remove করতে:

```
element.removeAttribute("disabled");
```

Example:

```
<button disabled>Submit</button>
```

```
const button = document.querySelector("button");  
button.removeAttribute("disabled");
```

এখন button enabled হতে পারে।

32. hasAttribute()

কোনো attribute আছে কি না check করতে:

```
if (button.hasAttribute("disabled")) {  
  console.log("Button is disabled");  
}
```

Return value:

true

অথবা:

false

33. `classList`

Element-এর CSS classes manipulate করার জন্য `classList` খুব useful।

HTML:

```
<div id="box" class="card"></div>
```

```
const box = document.querySelector("#box");  
  
console.log(box.classList);
```

34. `classList.add()`

নতুন class add:

```
box.classList.add("active");
```

আগে:

```
<div class="card"></div>
```

পরে:

```
<div class="card active"></div>
```

35. `classList.remove()`

Class remove:

```
box.classList.remove("active");
```

36. `classList.toggle()`

Class থাকলে remove করবে, না থাকলে add করবে।

```
box.classList.toggle("active");
```

এটি UI interaction-এর জন্য খুব useful।

Example:

```
button.addEventListener("click", () => {  
  menu.classList.toggle("open");  
});
```

এভাবে menu open/close করা যায়।

37. `classList.contains()`

কোনো class আছে কি না check করতে:

```
if (box.classList.contains("active")) {  
  console.log("Active");  
}
```

Return:

```
true
```

অথবা:

```
false
```

38. `classList` quick table

Method	কাজ
<code>add()</code>	class যোগ
<code>remove()</code>	class বাদ
<code>toggle()</code>	add/remove toggle
<code>contains()</code>	class আছে কিনা check

Mental model:

```
classList
├─ add
├─ remove
├─ toggle
└─ contains
```

39. `dataset`

HTML-এর custom `data-*` attributes access করার জন্য `dataset` ব্যবহার করা যায়।

HTML:

```
<button data-id="123" data-role="admin">
  Delete
</button>
```

JavaScript:

```
const button = document.querySelector("button");

console.log(button.dataset.id);
console.log(button.dataset.role);
```

Output:

```
123  
admin
```

40. dataset কেন useful?

Dynamic UI-তে element-এর সাথে ছোট metadata রাখতে এটি useful।

Example:

```
<button data-product-id="101">  
  Add to Cart  
</button>
```

JavaScript:

```
const button = document.querySelector("button");  
  
const productId = button.dataset.productId;  
  
console.log(productId);
```

Note:

```
data-product-id  
  ↓  
dataset.productId
```

41. DOM Traversal কী?

DOM-এর একটি element থেকে তার parent, child বা sibling element-এ যাওয়া হচ্ছে **DOM traversal**।

Example:

```
div  
├─ h2
```

```
└─ p
   └─ button
```

`div` থেকে আমরা:

```
parent
children
first child
next sibling
```

এগুলোর দিকে যেতে পারি।

42. `parentElement`

কোনো `element`-এর `parent element` পেতে:

```
const button = document.querySelector("button");

console.log(button.parentElement);
```

Example:

```
<div class="card">
  <button>Delete</button>
</div>
```

```
const button = document.querySelector("button");

console.log(button.parentElement);
```

Parent হবে:

```
<div class="card">
```


43. children

কোনো element-এর direct child elements পেতে:

```
const card = document.querySelector(".card");  
  
console.log(card.children);
```

Example:

```
<div class="card">  
  <h2>Product</h2>  
  <p>Price</p>  
  <button>Buy</button>  
</div>
```

children :

```
h2  
p  
button
```

44. firstElementChild

প্রথম child element:

```
const first = card.firstElementChild;
```

45. nextElementSibling

একটি element-এর পরের sibling element:

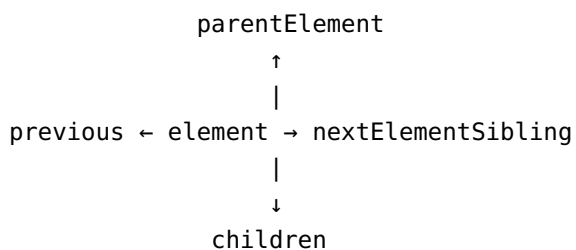
```
const title = document.querySelector("h2");  
  
console.log(title.nextElementSibling);
```

Example:

```
<h2>Product</h2>  
<p>Price</p>
```

Output হবে `<p>` element ।

46. Basic Traversal Mental Model



এগুলো DOM navigation-এর foundation ।

47. `createElement()`

JavaScript দিয়ে নতুন HTML element তৈরি করতে:

```
const paragraph = document.createElement("p");
```

এখন এটি memory-তে তৈরি হয়েছে, কিন্তু এখনো page-এ যোগ হয়নি ।

48. নতুন element-এর text set

```
const paragraph = document.createElement("p");  
paragraph.textContent = "Hello World";
```

49. নতুন element-এ class যোগ

```
paragraph.classList.add("text");
```

50. append()

নতুন element DOM-এ যোগ করতে:

```
document.body.append(paragraph);
```

এখন paragraph page-এ দেখা যাবে।

51. prepend()

Parent-এর শুরুতে element যোগ করতে:

```
document.body.prepend(paragraph);
```

Difference:

append()

→ শেষে যোগ

prepend()

→ শুরুতে যোগ

52. Existing element-এর ভিতর নতুন element

HTML:

```
<div id="container"></div>
```

JavaScript:

```
const container = document.querySelector("#container");

const heading = document.createElement("h2");

heading.textContent = "Hello";

container.append(heading);
```

Result:

```
<div id="container">
  <h2>Hello</h2>
</div>
```

53. remove()

কোনো element DOM থেকে remove করতে:

```
const paragraph = document.querySelector("p");

paragraph.remove();
```

Element আর DOM-এর অংশ থাকবে না।

54. Complete DOM Manipulation Example

HTML:

```
<div id="product">
  <h2>Old Product</h2>
  <p class="price">$50</p>
  <button>Buy</button>
</div>
```

JavaScript:

```
const product = document.querySelector("#product");

const title = product.querySelector("h2");
const price = product.querySelector(".price");
const button = product.querySelector("button");

title.textContent = "Laptop";
price.textContent = "$800";

button.classList.add("primary");

button.setAttribute("data-product-id", "101");
```

এখানে আমরা একসাথে ব্যবহার করেছি:

```
querySelector()
textContent
classList.add()
setAttribute()
```

55. Dynamic Element Creation Example

ধরো আমরা একটি product card JavaScript দিয়ে তৈরি করতে চাই।

```
const card = document.createElement("div");

card.classList.add("card");

const title = document.createElement("h2");

title.textContent = "Laptop";

const price = document.createElement("p");
```

```
price.textContent = "$800";  
  
card.append(title, price);  
  
document.body.append(card);
```

Result roughly:

```
<div class="card">  
  <h2>Laptop</h2>  
  <p>$800</p>  
</div>
```

56. `append()` একাধিক **element** নিতে পারে

```
card.append(title, price);
```

এটি valid।

এমনকি text-ও append করা যায়:

```
card.append("Hello");
```

57. `append()` vs `prepend()`

```
container.append(element);
```

মানে:

শেষে যোগ করে

আর:

```
container.prepend(element);
```

মানে:

শুরুতে যোগ করো

58. innerHTML বনাম createElement()

দুটো দিয়েই DOM তৈরি করা যায়।

innerHTML

```
container.innerHTML = `  
  <h2>Laptop</h2>  
  <p>$800</p>  
`;
```

createElement()

```
const title = document.createElement("h2");  
  
title.textContent = "Laptop";  
  
container.append(title);
```

কখন কোনটা?

Simple trusted static markup হলে:

innerHTML

convenient হতে পারে।

Dynamic বা untrusted content হলে:

```
createElement()  
textContent  
append()
```

অনেক ক্ষেত্রে safer এবং clearer ।

59. Important: `textContent` + `createElement()`

User input নিয়ে কাজ করলে:

```
const username = getUserInput();  
  
const p = document.createElement("p");  
  
p.textContent = username;  
  
container.append(p);
```

এটি user input-কে HTML হিসেবে interpret করবে না।

60. DOM Manipulation-এর Common Mistakes

Mistake 1 — `querySelector()` -এ `#` ভুলে যাওয়া

ভুল:

```
document.querySelector("title");
```

যদি ID হয়:

```
<h1 id="title">
```

সঠিক:

```
document.querySelector("#title");
```


Mistake 2 — `getElementById()` -এ `#` দেওয়া

ভুল:

```
document.getElementById("#title");
```

সঠিক:

```
document.getElementById("title");
```

Mistake 3 — `querySelector()` দিয়ে সব **element** পাওয়ার চেষ্টা

```
document.querySelector(".item");
```

শুধু first match ।

সবগুলো চাইলে:

```
document.querySelectorAll(".item");
```

Mistake 4 — `length()` লেখা

ভুল:

```
element.length();
```

সঠিক:

```
element.length;
```

`length` property ।

Mistake 5 — `element` না পাওয়ার পরও `manipulate` করা

```
const button = document.querySelector("#button");  
  
button.textContent = "Click";
```

যদি `button` না থাকে:

```
button === null
```

তাহলে `error` হবে।

Safe:

```
if (button) {  
  button.textContent = "Click";  
}
```

Mistake 6 — `innerHTML` দিয়ে `untrusted data inject` করা

Avoid:

```
element.innerHTML = userInput;
```

যদি `userInput` `untrusted` হয়।

Prefer:

```
element.textContent = userInput;
```

Mistake 7 — `className` দিয়ে পুরো `class replace` করা

```
element.className = "active";
```

এতে `existing classes` হারিয়ে যেতে পারে।

যদি শুধু একটি class যোগ করতে চাও:

```
element.classList.add("active");
```

61. DOM Selection Quick Comparison

Method	কাজ	Return
<code>querySelector()</code>	first matching element	Element / <code>null</code>
<code>querySelectorAll()</code>	all matching elements	NodeList
<code>getElementById()</code>	ID দিয়ে element	Element / <code>null</code>
<code>getElementsByClassName()</code>	class দিয়ে elements	HTMLCollection

62. Content Manipulation Quick Comparison

Property	Main use
<code>innerHTML</code>	HTML + content
<code>textContent</code>	plain text
<code>innerText</code>	visible/rendered text

63. Attribute Quick Comparison

```
setAttribute()
```

→ set

```
getAttribute()
```

→ read

```
removeAttribute()
```

→ remove

```
hasAttribute()
```

→ check

64. Class Manipulation Quick Comparison

```
element.classList.add("active");
```

→ add

```
element.classList.remove("active");
```

→ remove

```
element.classList.toggle("active");
```

→ toggle

```
element.classList.contains("active");
```

→ check

65. DOM Creation Quick Flow

নতুন element তৈরি করার common pattern:

```
const element = document.createElement("div");  
  
element.classList.add("card");  
  
element.textContent = "Hello";
```

```
parent.append(element);
```

Mental model:

```
create
  ↓
configure
  ↓
append
```

66. Real-World Example — Todo Item

HTML:

```
<div id="todo-list"></div>
```

JavaScript:

```
const todoList = document.querySelector("#todo-list");

const todo = document.createElement("div");

todo.classList.add("todo");

todo.textContent = "Learn JavaScript DOM";

todoList.append(todo);
```

Result:

```
<div id="todo-list">
  <div class="todo">
    Learn JavaScript DOM
  </div>
</div>
```

67. Real-World Example — Dynamic Product Card

```
function createProduct(name, price) {
  const card = document.createElement("div");

  card.classList.add("product-card");

  const title = document.createElement("h2");

  title.textContent = name;

  const priceElement = document.createElement("p");

  priceElement.textContent = `$$${price}`;

  card.append(title, priceElement);

  return card;
}

const product = createProduct("Laptop", 800);

document.body.append(product);
```

এখানে DOM manipulation-কে function-এর মধ্যে reusable করা হয়েছে।

68. DOM Manipulation + Data

ধরো API থেকে data এসেছে:

```
const products = [
  {
    name: "Laptop",
    price: 800
  },
  {
    name: "Mouse",
    price: 30
  }
];
```

আমরা dynamically card তৈরি করতে পারি:

```
const container = document.querySelector("#products");

products.forEach((product) => {
  const card = document.createElement("div");

  const title = document.createElement("h2");

  title.textContent = product.name;

  const price = document.createElement("p");

  price.textContent = `$$${product.price}`;

  card.append(title, price);

  container.append(card);
});
```

এটি DOM-এর একটি খুব common real-world pattern ।

69. DOM vs React

তুমি যেহেতু React শিখছো, এখানে একটি important connection আছে।

Vanilla JavaScript:

```
const title = document.querySelector("#title");

title.textContent = "Hello";
```

React-এ সাধারণত তুমি সরাসরি DOM manipulate করবে না।

React:

```
<h1>{title}</h1>
```

State পরিবর্তন হলে React নিজেই UI update করে।

তাই:

Vanilla JS
→ manually manipulate DOM

React
→ state/props change
→ React updates DOM

তবুও DOM জানা React developer-এর জন্য important, কারণ:

- browser কীভাবে কাজ করে বুঝতে
- events বুঝতে
- `ref` বুঝতে
- performance বুঝতে
- debugging করতে
- third-party DOM libraries বুঝতে

70. গুরুত্বপূর্ণ Mental Model

DOM manipulation-কে চারটি step হিসেবে ভাবো:

1. Find
↓
2. Read / Change
↓
3. Create / Update
↓
4. Insert / Remove

Example:

```
const button = document.querySelector("#button");  
  
button.textContent = "Submit";  
  
button.classList.add("primary");  
  
button.setAttribute("disabled", "");  
  
button.remove();
```


71. Quick Revision

Selection

```
document.querySelector()  
document.querySelectorAll()  
document.getElementById()  
document.getElementsByClassName()
```

Content

```
element.innerHTML  
element.textContent  
element.innerText
```

Attributes

```
element.setAttribute()  
element.getAttribute()  
element.removeAttribute()  
element.hasAttribute()
```

Classes

```
element.classList.add()  
element.classList.remove()  
element.classList.toggle()  
element.classList.contains()
```

Traversal

```
element.parentElement  
element.children  
element.firstElementChild  
element.nextElementSibling
```

Creation

```
document.createElement()  
element.append()
```

```
element.prepend()  
element.remove()
```

Data attributes

```
element.dataset
```

72. Practice Questions

Practice 1

HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript দিয়ে text `"Welcome"` করো।

Practice 2

সব `.card` element select করে প্রত্যেকটির background-related class `active` add করো।

Practice 3

একটি button-এর `disabled` attribute আছে কিনা check করো।

Practice 4

একটি `<div>` তৈরি করে:

```
Hello World
```

text হিসেবে সেট করো এবং `body` -তে append করো।

Practice 5

একটি element-এর class `active` থাকলে remove করবে, না থাকলে add করবে।

Practice 6

HTML:

```
<div id="box">  
  <h2>Hello</h2>  
</div>
```

JavaScript দিয়ে `<h2>` -এর text:

```
JavaScript DOM
```

করো।

Practice 7

একটি button-এর:

```
data-product-id="123"
```

থেকে `123` value বের করো।

73. Interview Questions

1. DOM কী?

Answer:

DOM হলো browser-এর তৈরি object-based representation of an HTML document, যেটা JavaScript দিয়ে access ও manipulate করা যায়।

2. `querySelector()` এবং `querySelectorAll()` এর difference কী?

```
querySelector()  
→ first matching element  
  
querySelectorAll()  
→ all matching elements
```

3. `querySelectorAll()` কী return করে?

সাধারণত একটি **static NodeList** return করে।

4. `getElementsByClassName()` কী return করে?

একটি **HTMLCollection** return করে, যা সাধারণত live collection।

5. `innerHTML` এবং `textContent` এর difference কী?

```
innerHTML  
→ HTML parse করে  
  
textContent  
→ plain text হিসেবে কাজ করে
```

6. `innerText` এবং `textContent` এর difference কী?

`innerText` rendered/visible text-এর সাথে বেশি সম্পর্কিত, আর `textContent` DOM-এর text content নিয়ে কাজ করে।

7. কেন `textContent` user input-এর ক্ষেত্রে safer হতে পারে?

কারণ এটি input-কে HTML হিসেবে parse করে না।

8. `classList.toggle()` কী করে?

Class থাকলে remove করে, না থাকলে add করে।

9. dataset কী?

HTML-এর `data-*` custom attributes access করার convenient API।

10. HTMLCollection এবং NodeList-এর একটি গুরুত্বপূর্ণ difference কী?

HTMLCollection সাধারণত live, কিন্তু `querySelectorAll()` থেকে পাওয়া NodeList সাধারণত static।

74. Golden Rules

Rule 1: প্রথম matching element চাইলে `querySelector()`।

Rule 2: সব matching element চাইলে `querySelectorAll()`।

Rule 3: ID-এর জন্য `getElementById()` straightforward।

Rule 4: Plain text-এর জন্য `textContent` prefer করো।

Rule 5: Untrusted input সরাসরি `innerHTML`-এ দিও না।

Rule 6: Class manipulate করতে `classList` ব্যবহার করো।

Rule 7: Attribute manipulate করতে `setAttribute()`, `getAttribute()`, `removeAttribute()` জানো।

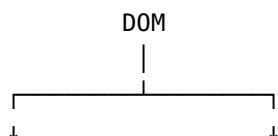
Rule 8: Dynamic element তৈরির foundation হলো `createElement()` + `append()`।

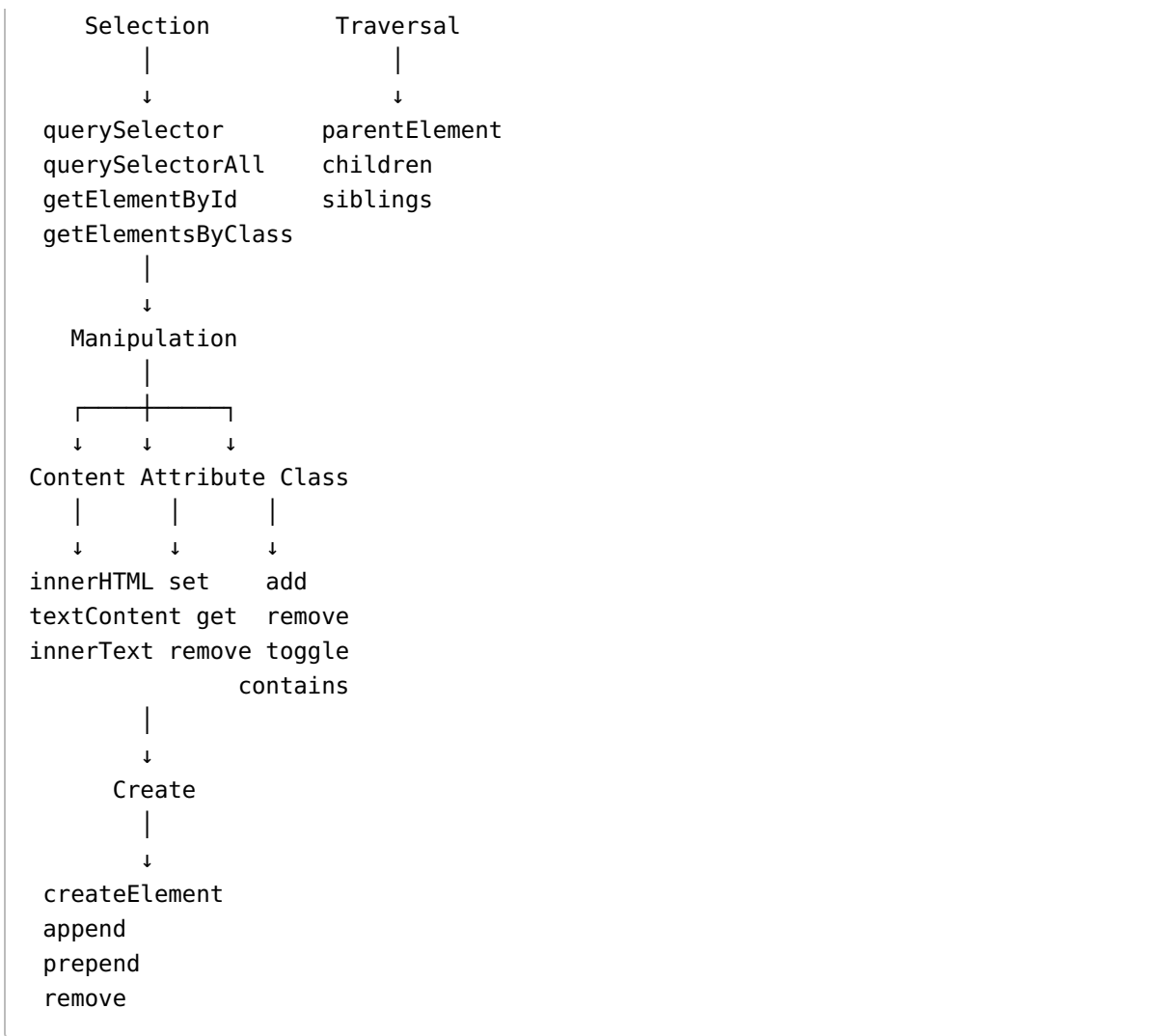
Rule 9: `querySelectorAll()` → NodeList, `getElementsByClassName()` → HTMLCollection।

Rule 10: DOM manipulation-এর আগে element সত্যিই পাওয়া গেছে কিনা check করা গুরুত্বপূর্ণ।

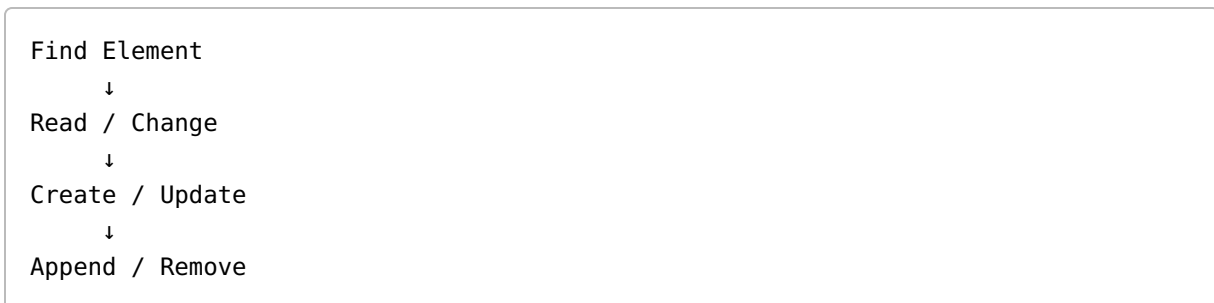
75. Final Mental Model

পুরো DOM Selection & Manipulation-কে এভাবে মনে রাখো:





সবচেয়ে important flow:



এই foundation ভালোভাবে বুঝলে পরের topic **DOM Events & Event Propagation** অনেক সহজ হবে।

✓ Section 13.1 Mastery Checklist

- [] DOM কী বুঝি
- [] DOM Tree বুঝি
- [] `document` কী বুঝি
- [] `querySelector()` পারি
- [] `querySelectorAll()` পারি
- [] `NodeList` বুঝি
- [] `getElementById()` পারি
- [] `getElementsByClassName()` পারি
- [] `HTMLCollection` বুঝি
- [] Live vs static collection বুঝি
- [] `innerHTML` বুঝি
- [] `textContent` বুঝি
- [] `innerText` বুঝি
- [] XSS-এর basic ধারণা আছে
- [] `setAttribute()` পারি
- [] `getAttribute()` পারি
- [] `removeAttribute()` পারি
- [] `hasAttribute()` পারি
- [] `classList.add()` পারি
- [] `classList.remove()` পারি
- [] `classList.toggle()` পারি
- [] `classList.contains()` পারি
- [] `dataset` বুঝি
- [] `parentElement` বুঝি
- [] `children` বুঝি
- [] `firstElementChild` বুঝি
- [] `nextElementSibling` বুঝি
- [] `createElement()` পারি
- [] `append()` পারি
- [] `prepend()` পারি
- [] `remove()` পারি
- [] Dynamic DOM element তৈরি করতে পারি
- [] `innerHTML` vs `textContent` explain করতে পারি
- [] `NodeList` vs `HTMLCollection` explain করতে পারি
- [] Vanilla DOM manipulation এবং React-এর approach-এর difference বুঝি

🔥 Section 13.1-এর Core Formula

SELECT
→ `querySelector` / `querySelectorAll`

READ

→ `textContent` / `innerText` / `getAttribute`

CHANGE

→ `textContent` / `innerHTML` / `setAttribute`

CLASS

→ `classList.add` / `remove` / `toggle` / `contains`

TRAVERSE

→ `parent` / `children` / `siblings`

CREATE

→ `createElement`

INSERT

→ `append` / `prepend`

REMOVE

→ `remove`

এগুলো **DOM-এর core building blocks**।